

Dokumentacja końcowa

Celem projektu jest implementacja interpretera własnego języka ogólnego przeznaczenia. Język ten będzie uproszczeniem języka C.

1. Założenia:

- Język obsługuje typy liczbowe *int* i *float* oraz pozwala na wykonywanie operacji na liczbach.
- Język obsługuje typ *string* i pozwala na konkatencję ciągów znaków.
- Język obsługuje typ logiczny *bool*.
- Komentarze rozpoczyna znak *#*.
- Język pozwala na tworzenie zmiennych o statycznym, słabym typowaniu. Domyślnie zmienne mają stałą wartość po jej przypisaniu.
- Język pozwala na tworzenie własnych funkcji o parametrach przekazywanych przez wartość. Funkcje mogą, ale nie muszą zwracać wartości (*void*).
- Możliwe jest rekursywne wywołanie funkcji.
- Nie jest możliwe przeciążanie funkcji.
- Dozwolone typy funkcji: *void*, *int*, *float*, *string*, *bool*.
- Dozwolone typy parametrów funkcji: *int*, *float*, *string*, *bool*, rekord wariantowy.
- Język obsługuje instrukcję pętli *while* oraz instrukcję warunkową *if-else*.
- Możliwe jest wypisanie tekstu na ekranie za pomocą funkcji *print()*.
- Język obsługuje struktury i rekord wariantowy (rekord ten ma budowę podobną do struktury, ale zawiera dodatkowe pole informujące o typie aktualnie przechowywanej wartości). Polem struktury może być inna struktura lub rekord wariantowy. Jednym z typów rekordu wariantowego również może być rekord wariantowy.
- Każdy program musi zawierać funkcję *main()*.

2. Struktura projektu

Projekt został zaimplementowany w języku C++. Składa się z trzech części:

- lekser - przetwarza kod źródłowy na odpowiednie tokeny
- parser - buduje drzewo programu z tokenów wygenerowanych przez lekser według gramatyki języka
- interpreter - przechodzi po drzewie wygenerowanym przez parser i wykonuje kolejne instrukcje

3. Przykładowy kod:

- prosta funkcja

```
int fun(int a)
[
    a = a + 1;
    return a;
]

int main()
[
    mut int a = 2;
    mut int b = fun(a);
    print(a);
    print(b);
    return 0;
]

# wyświetli się:
# 2
# 3
```

- funkcja z rekursją

```
int silnia(int n)
[
    if (n == 0)
```

```

    [
        return 1;
    ]
else
    [
        return n * silnia(n-1);
    ]
]

```

- funkcja *main()* z pętlą *while()*

```

int main()
[
    int a = 10;
    mut int b = 1;
    b = silnia(3);
    while (b < a)
    [
        print(b);
        b = b + 1;
    ]
    return 0;
]

```

- konkatencja stringów

```

string x = "hello";
string y = "world";
string z = x + y;

```

- konwersja typu

```

string s = 12.3;
string t = " meters";
string u = s + t;

```

- struktura

```
struct Item
[
    string name;
    mut float value;
]

new Item my_item("bread", 3.45);
my_item.value = 4.56;
```

- rekord variantowy

```
variant[int, string] a;
a = "hello";

match a
[
    int i
    [
        print("Wartosc typu int: \n");
        print(i);
    ]
    string i
    [
        print("Wartosc typu string: \n");
        print(i);
    ]
    default
    [
        print("Wartosc innego typu");
    ]
]
```

4. Obsługa błędów

Każdy błąd skutkuje zatrzymaniem programu.

Przykłady błędów:

- brak średnika

```
bool complete = true
```

Komunikat: “Error in line x, column y: missing a semicolon”

- próba zmiany wartości zmiennej o stałej wartości:

```
int x = 5;  
x = 3;
```

Komunikat: “Error in line x, column y: cannot modify a const value”

- próba użycia niezainicjowanej zmiennej

```
int x = y;
```

Komunikat: “Error in line x, column y: uninitialized variable”

- dzielenie przez 0

```
int div(int a, int b)  
{  
    return a/b;  
}  
  
int main()  
{  
    int c = div(2, 0);  
}
```

```
    print(c);  
    return 0;  
]
```

Komunikat: “Error in line x, column y: cannot divide by 0”

- nieprawidłowy typ

```
int a = "hello";  
print(a);
```

Komunikat: “Error in line x, column y: incorrect type”

5. Kompilacja i uruchomienie

```
mkdir build  
cd build  
cmake ..  
make  
./main [path_to_file]  
np.  
./main ../examples/silnia.txt
```

6. Testy

Do każdej części projektu zostały napisane testy jednostkowe:

- *LexerTests* - sprawdzają, czy lekser prawidłowo tworzy zarówno pojedyncze tokeny, jak i ciągi tokenów
- *ParserTests* - sprawdzają, czy parser prawidłowo parsuje produkcje gramatyki, a także czy zgłasza błąd w przypadku nieprawidłowej sekwencji tokenów
- *InterpreterTests* - sprawdzają, czy wynik działania programu jest prawidłowy, a także czy wychwytywane są błędy semantyczne (np. nieprawidłowy typ argumentu funkcji)

Aby uruchomić poszczególne testy, należy po skompilowaniu wykonać odpowiednie polecenie:

```
./LexerTests  
./ParserTests  
./InterpreterTests
```

7. Gramatyka

```
digit_not_zero    =    '1' | '2' | '3' | '4' | '5'  
                  |    '6' | '7' | '8' | '9';  
  
digit             =    '0'  
                  |    digit_not_zero;  
  
letter            =    'a' | 'b' | ... | 'z'  
                  |    'A' | 'B' | ... | 'Z';  
  
symbol            =    '\' | \. | \, | \/ | \'?  
                  |    '<' | '>' | ';' | ':' | '!'  
                  |    '@' | '#' | '$' | '%' | '^'  
                  |    '&' | '*' | '(' | ')' | '_'  
                  |    '-' | '+' | '=';  
  
escaped_char      =    '\\', ('\\' | '\"' | 'n' | 't');  
  
type              =    'bool'  
                  |    'int'  
                  |    'float'  
                  |    'string';  
  
identifier        =    letter, {letter | digit | '_'};  
  
bool              =    'true'  
                  |    'false';  
  
int               =    '0'  
                  |    digit_not_zero , {digit};  
  
float             =    int, \., {digit};
```

```

string          =    '\'',
                    {letter | digit | symbol | escaped_char},
                    '\';

literal         =    bool
                    |    int
                    |    float
                    |    string;

argument_list   =    [expression, {'\,', expression}];

field_or_fun_call =    identifier,
                        ['(', [argument_list], ')']
                        | {'\.', identifier}];

term            =    ['!' | '-' ],
                    (field_or_fun_call
                     | '(' , expression, ') '
                     | literal);

mult_expression =    term, {'*', ('/' ), term};

add_expression  =    mult_expression, {'+' | '-' },
                    mult_expression;

comparison      =    add_expression,
                    ['==' | '!=' | '>' | '<' | '>=' | '<=' ),
                    add_expression];

and_expression   =    comparison, {'and', comparison};

expression      =    and_expression, {'or', and_expression};

match_case      =    (type | variant), identifier, block
                    | 'default', block;

match_statement =    'match', expression, '[', {match_case},
                    '];

struct_creation  =    'new', identifier, identifier, '(',
                    argument_list, ')', ';';

struct_field     =    ['mut'], type, identifier, ';'
                    | identifier, identifier, ';'
                    | variant_declaration;

```



```

struct_field_list    =    struct_field, {struct_field};

struct_declaration  =    'struct', identifier, '[',
                        struct_field_list, ']';

variant             =    'variant', '[',
                        (type | variant), {type | variant}, ']';

variant_declaration =    variant, identifier, ';';

return_statement    =    'return', [expression], ';';

while_statement     =    'while', '(', expression, ')', block;

if_statement        =    'if', '(', expression, ')', block,
                        ['else', block];

assignment          =    field_or_fun_call, ['=', expression], ';';

variable_declaration =    ['mut'], type, identifier, ['=',
                        expression], ';';

parameter           =    (type | variant), identifier;

parameter_list      =    [parameter, {'', parameter}];

statement           =    variable_declaration
                        |
                        assignment
                        |
                        if_statement
                        |
                        while_statement
                        |
                        return_statement
                        |
                        variant_declaration
                        |
                        struct_declaration
                        |
                        struct_creation
                        |
                        match_statement;

block               =    '[', {statement}, ']';

function_declaration =    (type | 'void'), identifier, '(',
                        parameter_list, ')', block;

declaration         =    function_declaration
                        |
                        variable_declaration
                        |
                        variant_declaration
                        |
                        struct_declaration

```

```
program          =      {declaration};
```